

**UNIVERSIDADE FEDERAL DE UBERLÂNDIA**

PPGEELT – Pós-Graduação em Engenharia Elétrica

---

LUCAS ALBINO MARTINS

12622EEL006

**TRABALHO 05 - CLASSIFICAÇÃO DE PADRÕES COM A  
REDE NEURAL ADALINE**

Disciplina: EL056 - Redes Neurais Artificiais

Uberlândia

2026

**UNIVERSIDADE FEDERAL DE UBERLÂNDIA**

PPGEELT – Pós-Graduação em Engenharia Elétrica

---

LUCAS ALBINO MARTINS

12622EEL006

**TRABALHO 05 - CLASSIFICAÇÃO DE PADRÕES COM A  
REDE NEURAL ADALINE**

Disciplina: EL056 - Redes Neurais Artificiais

Trabalho apresentado ao Programa de Pós-Graduação em Engenharia Elétrica (PPGEELT) da Universidade Federal de Uberlândia como requisito parcial de avaliação da disciplina.

Disciplina: EL056 - Redes Neurais Artificiais.

Professor: Prof. Dr. Keiji Yamanaka.

Uberlândia

2026

# Treinamento e avaliação de uma rede Adaline na classificação de padrões da base de dados B2

Lucas Albino Martins – 12622EEL006

---

**Resumo** — Este trabalho apresenta o treinamento e a avaliação de uma rede neural Adaline aplicada à classificação de padrões da base de dados B2, composta por 20 amostras descritas por dois atributos contínuos e rotuladas em duas classes bipolares. A rede foi treinada pela regra Delta (Widrow–Hoff), com taxa de aprendizagem  $\eta = 0,0025$  e critério de parada baseado na variação do erro quadrático médio ( $\varepsilon = 10^{-6}$ ). O erro quadrático total decresceu monotonicamente de 28,26 para 3,71 ao longo de 704 épocas, a 0,139 % do mínimo analítico, e a rede classificou corretamente os 20 padrões. O teste empregou *holdout* estratificado 70/30 repetido dez vezes ( $98,3 \% \pm 5,3 \%$ ), validação cruzada *leave-one-out* (100 %) e operação sobre padrões inéditos. Embora Adaline e Perceptron atinjam 100 % de acerto, a primeira produz fronteira com margem 97 vezes maior, reproduzível e mais robusta a ruído nos atributos.

**Palavras-chave:** Adaline; regra Delta; classificação de padrões; erro quadrático total; Widrow–Hoff.

## 1 Introdução

A Adaline (*ADaptive Linear NEuron*), proposta por Widrow e Hoff em 1960, é uma das arquiteturas fundadoras da área de redes neurais artificiais [1]. Contemporânea do Perceptron de Rosenblatt e igualmente limitada a problemas linearmente separáveis, ela introduziu uma contribuição conceitual muito mais duradoura: o treinamento por minimização de uma função de custo contínua e diferenciável, por descida do gradiente — a regra Delta, ou algoritmo LMS (*Least Mean Square*), base direta do *backpropagation* [4, 2].

O objetivo deste trabalho é treinar uma Adaline para classificar os padrões da base B2, apresentar graficamente o erro quadrático total (EQT) durante o treinamento e avaliar a rede sobre dados não utilizados no ajuste dos pesos. Além desses três itens, investigam-se a influência da taxa de aprendizagem, a estabilidade da solução frente a diferentes inicializações e as diferenças práticas entre as soluções da Adaline e do Perceptron.

## 2 Fundamentação teórica

A Adaline é uma rede de camada única constituída por um único neurônio. Adotando a convenção de Silva, Spatti e Flauzino [3], na qual o vetor de entrada é aumentado com  $x_0 = -1$  para incorporar o limiar de ativação  $\theta$  ao vetor de pesos, o potencial de ativação é

$$u = \sum_{i=1}^n w_i x_i - \theta = \mathbf{W}^\top \mathbf{X}, \quad \mathbf{W} = [\theta \ w_1 \ \cdots \ w_n]^\top. \quad (1)$$

Durante o treinamento a função de ativação é **linear**,  $g(u) = u$ ; somente na operação aplica-se a função sinal, produzindo  $y = \text{sgn}(u) \in \{-1, +1\}$ . Essa separação é a diferença essencial em relação ao Perceptron, que calcula o erro *após* a função degrau e descarta a informação sobre

quão distante da fronteira o padrão se encontra. O ajuste minimiza o erro quadrático entre o valor desejado  $d^{(p)}$  e o potencial  $u^{(p)}$  sobre os  $P$  padrões:

$$E(\mathbf{W}) = \frac{1}{2} \sum_{p=1}^P (d^{(p)} - u^{(p)})^2 \quad \implies \quad \nabla E(\mathbf{W}) = - \sum_{p=1}^P (d^{(p)} - u^{(p)}) \mathbf{X}^{(p)}. \quad (2)$$

Como  $E(\mathbf{W})$  é uma forma quadrática nos pesos, sua superfície é convexa e possui mínimo global único. Caminhando na direção oposta ao gradiente obtém-se a regra Delta, aqui aplicada de forma sequencial:

$$\mathbf{W} \leftarrow \mathbf{W} + \eta (d^{(p)} - u^{(p)}) \mathbf{X}^{(p)}. \quad (3)$$

O treinamento é monitorado pelo erro quadrático total e pelo médio ao final de cada época  $k$ ,

$$\text{EQT}(k) = \sum_{p=1}^P (d^{(p)} - u^{(p)})^2, \quad \text{EQM}(k) = \frac{1}{P} \text{EQT}(k), \quad (4)$$

com critério de parada  $|\text{EQM}(k) - \text{EQM}(k-1)| \leq \varepsilon$ . Por comparar alvos bipolares a uma saída contínua, o EQT converge para valor estritamente positivo mesmo com separação perfeita: o patamar atingido indica proximidade do mínimo, e não a quantidade de padrões classificados incorretamente.

### 3 Metodologia

A base B2 reúne 20 amostras descritas por dois atributos contínuos,  $s_1$  e  $s_2$ , associadas a um rótulo bipolar  $t \in \{-1, +1\}$ , perfeitamente balanceada (10 amostras por classe). Ambos os atributos variam em faixas equivalentes —  $s_1 \in [0,169; 2,764]$  e  $s_2 \in [0,090; 2,932]$  —, dispensando normalização. Na Figura 1(a) as classes ocupam regiões opostas do plano e aparentam ser linearmente separáveis, com os padrões nº 13 (0,856; 1,904) e nº 15 (1,587; 1,642) próximos da fronteira.

A rede foi implementada em Python 3, com NumPy, pandas e Matplotlib. Os pesos iniciais foram sorteados de uma distribuição uniforme em  $[-0,5; 0,5]$  e o treinamento seguiu a Equação 3, com  $\eta = 0,0025$ ,  $\varepsilon = 10^{-6}$  e limite de segurança de 10.000 épocas. Como avaliar a rede apenas sobre os padrões usados no ajuste produziria estimativa otimista, o teste empregou três procedimentos complementares: *holdout* estratificado 70/30 (14 amostras para treinamento e 6 para teste), repetido em dez partições aleatórias; validação cruzada *leave-one-out*, em que cada amostra é reservada enquanto a rede é retreinada com as 19 restantes; e operação sobre seis padrões inéditos.

## 4 Resultados e discussão

### 4.1 Treinamento e erro quadrático total

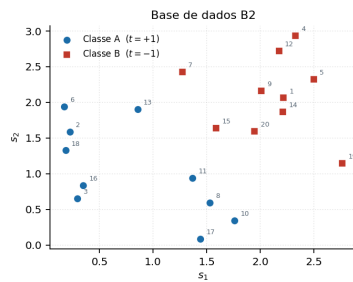
O treinamento sobre a base completa convergiu em **704 épocas**, com pesos finais  $\theta = -2,0418$ ,  $w_1 = -0,7492$  e  $w_2 = -0,6202$  (Tabela 1), correspondentes à fronteira de decisão

$$-0,7492 s_1 - 0,6202 s_2 + 2,0418 = 0. \quad (5)$$

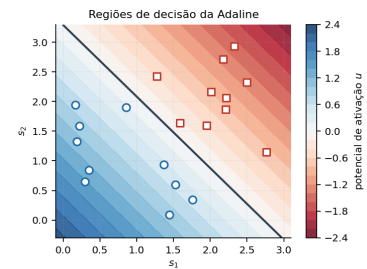
**Tabela 1:** Parâmetros e resultados do treinamento sobre a base B2 completa.

| Parâmetro                        | Valor                    | Resultado               | Valor    |
|----------------------------------|--------------------------|-------------------------|----------|
| Taxa de aprendizagem $\eta$      | 0,0025                   | Épocas até convergir    | 704      |
| Precisão requerida $\varepsilon$ | $10^{-6}$                | EQT inicial             | 28,2577  |
| Padrões de treinamento $P$       | 20                       | EQT final               | 3,7098   |
| Entradas por padrão $n$          | 2                        | EQM final               | 0,185491 |
| Pesos iniciais                   | $\mathcal{U}(-0,5; 0,5)$ | Acurácia no treinamento | 100,0 %  |

A Figura 2 apresenta a evolução do erro quadrático total exigida no enunciado. A curva é **monotonicamente decrescente e livre de oscilações**, comportamento esperado de uma descida de gradiente sobre superfície convexa: cerca de 90 % da redução ocorre nas primeiras 200 épocas, e as demais correspondem ao refinamento fino interrompido pelo critério de parada, explicitado no painel (b). O patamar em  $\text{EQT} \approx 3,71$  *não* representa erro de classificação: é o resíduo estrutural entre alvos bipolares e saída linear, e nesse ponto a rede já acerta os 20 padrões. A Figura 1(b) exhibe as regiões de decisão coloridas pelo valor de  $u$  — informação que o Perceptron descarta, mas que na Adaline mede a confiança da decisão.

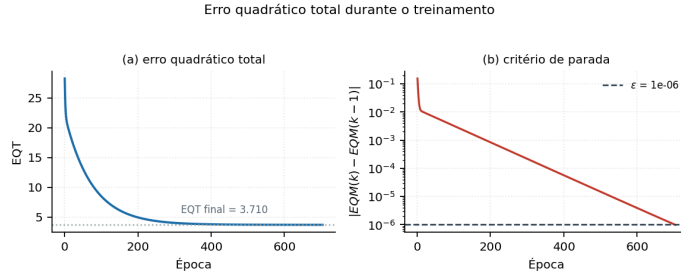


(a) padrões da base B2



(b) regiões de decisão

**Figura 1:** Distribuição das classes da base B2 e regiões de decisão obtidas pela Adaline treinada.



**Figura 2:** Erro quadrático total ao longo do treinamento (a) e variação do erro quadrático médio entre épocas consecutivas, usada como critério de parada (b).

## 4.2 Influência da taxa de aprendizagem

Como a função de custo é quadrática, o mínimo desta base tem solução fechada pelas equações normais:  $\mathbf{W}^* = [-2,0665; -0,7504; -0,6251]$ , com  $\text{EQM}^* = 0,185234$ . Essa referência permite medir exatamente aonde cada treinamento chegou — a configuração adotada ficou a apenas 0,139 % desse valor.

A Tabela 2 resume cinco treinamentos idênticos exceto pelo valor de  $\eta$ . A leitura imediata é que taxas menores exigem mais épocas; o dado relevante, porém, é que taxas maiores *param num erro pior*. A explicação natural seria divergência, mas ela não se sustenta: o critério de estabilidade do LMS é  $\eta < 2/\lambda_{\max}$ , e os autovalores da matriz de autocorrelação desta base ( $\lambda_{\max} = 6,2583$ ) situam o limite em  $\eta < 0,3196$ , acima de todos os valores testados.

O fenômeno é o **desajuste** (*misadjustment*) do LMS sequencial: dentro de uma mesma época cada padrão puxa os pesos para o seu lado e, com passo grande, a composição das  $P$  atualizações possui um ponto fixo *deslocado* do mínimo verdadeiro, com desvio proporcional a  $\eta$ . Que se trata de ponto fixo, e não de oscilação aleatória, comprova-se pelo fato de  $\eta = 0,1$  convergir para pesos idênticos a partir de inicializações diferentes. O critério de parada, portanto, não falhou — a variação entre épocas realmente foi a zero; o que compromete o resultado é a localização do ponto de convergência, cujo desvio inclina a fronteira o bastante para deixar o padrão nº 13 do lado errado ( $u = -0,296$ ). Adotou-se  $\eta = 0,0025$  por retorno decrescente: entre ele e 0,001 o EQM melhora apenas na quarta casa decimal, ao custo do dobro de épocas.

**Tabela 2:** Efeito da taxa de aprendizagem sobre a convergência ( $\epsilon = 10^{-6}$ , mesma inicialização de pesos), comparado ao mínimo obtido analiticamente.

| Taxa $\eta$             | Épocas     | EQT final     | EQM final       | Acurácia       |
|-------------------------|------------|---------------|-----------------|----------------|
| 0,1000                  | 44         | 10,1977       | 0,509886        | 95,0 %         |
| 0,0500                  | 75         | 6,5270        | 0,326348        | 95,0 %         |
| 0,0100                  | 187        | 3,7796        | 0,188979        | 100,0 %        |
| <b>0,0025</b>           | <b>704</b> | <b>3,7098</b> | <b>0,185491</b> | <b>100,0 %</b> |
| 0,0010                  | 1586       | 3,7090        | 0,185450        | 100,0 %        |
| <i>mínimo analítico</i> | —          | 3,7047        | 0,185234        | 100,0 %        |

### 4.3 Teste da rede treinada

A rede do trabalho é a treinada com as 20 amostras (Seção 4.1). Como a base B2 não acompanha conjunto de teste separado, os protocolos a seguir **retreinam** a rede em subconjuntos, apenas para estimar a capacidade de generalização.

Na primeira partição *holdout*, a rede treinada com 14 amostras classificou corretamente as 6 reservadas (Tabela 3 e Figura 3(a)). Em dez partições distintas, a acurácia média de teste foi de **98,3 %  $\pm$  5,3 %**: nove rodadas com 100 % e uma com 83,3 % (5 acertos em 6). O único erro do experimento envolveu o padrão nº 13, com  $u = -0,057$  — praticamente sobre a fronteira — cuja ausência do treinamento deslocou o hiperplano o bastante para inverter sua classificação.

**Tabela 3:** Teste da rede treinada com 14 amostras: saídas para as 6 amostras reservadas ( $\theta = -2,3313$ ,  $w_1 = -0,8215$ ,  $w_2 = -0,7357$ ).

| Amostra | $s_1$ | $s_2$ | $t$ (desejado) | $u$     | $y$ (obtido) | Resultado |
|---------|-------|-------|----------------|---------|--------------|-----------|
| 11      | 1,367 | 0,938 | +1             | +0,5181 | +1           | correto   |
| 3       | 0,294 | 0,651 | +1             | +1,6108 | +1           | correto   |
| 1       | 2,215 | 2,063 | -1             | -1,0062 | -1           | correto   |
| 19      | 2,764 | 1,149 | -1             | -0,7847 | -1           | correto   |
| 4       | 2,327 | 2,932 | -1             | -1,7376 | -1           | correto   |
| 16      | 0,350 | 0,840 | +1             | +1,4257 | +1           | correto   |

A validação *leave-one-out*, mais adequada a bases pequenas por usar 19 das 20 amostras em cada treinamento, resultou em **20 acertos em 20 partições (100 %)**, inclusive para o padrão nº 13. O contraste é informativo: com 14 amostras a fronteira ainda é sensível à composição do conjunto de treinamento; com 19, já se estabiliza. Os seis padrões inéditos receberam rótulos coerentes com a Equação 5, com  $|u|$  proporcional à distância até a fronteira: (0,50; 0,50) produziu  $u = +1,3571$ , enquanto (1,50; 1,20) resultou em  $u = +0,1737$ , decisão de baixa confiança.

### 4.4 Estabilidade e comparação com o Perceptron

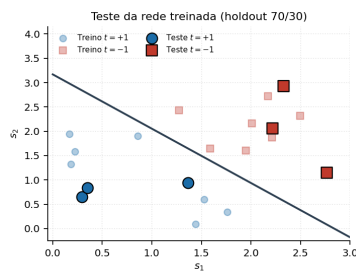
Com 20 inicializações aleatórias distintas, os parâmetros finais mantiveram-se praticamente idênticos ( $\theta = -2,0418 \pm 4 \times 10^{-5}$ ; EQM =  $0,185492 \pm 2,9 \times 10^{-7}$ ), variando apenas o número de épocas ( $686,9 \pm 17,8$ ) — confirmação empírica da unicidade do mínimo.

O contraste com o Perceptron treinado sobre a mesma base evidencia o efeito prático da regra Delta. O Perceptron convergiu em apenas 23 épocas, também com 100 % de acerto, porque interrompe o aprendizado assim que nenhum padrão é classificado incorretamente — a primeira fronteira válida é aceita, e ela passa a 0,0018 do padrão nº 13. A Adaline, por minimizar o erro sobre o potencial de ativação, segue ajustando os pesos mesmo sem erros e posiciona o hiperplano a 0,1703 do padrão mais próximo — margem **97 vezes maior**, com as fronteiras separadas por 16,92°.

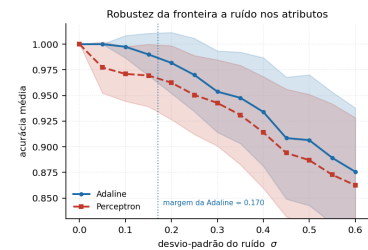
## 4.5 Robustez a ruído nos atributos

A vantagem da margem é, até aqui, um argumento geométrico. Para verificá-la, as duas redes já treinadas foram submetidas a entradas corrompidas por ruído gaussiano de desvio-padrão  $\sigma$  crescente, com 200 repetições por nível (Figura 3(b)). Já em  $\sigma = 0,05$  a Adaline permanece em 100 % enquanto o Perceptron cai para 97,7 % — o padrão nº 13 sendo invertido em cerca de metade das repetições, por estar praticamente sobre a sua fronteira. A vantagem persiste em toda a faixa testada, entre 1,0 e 2,6 pontos percentuais.

A diferença, porém, não cresce indefinidamente: com  $\sigma$  elevado o ruído desloca também padrões distantes da fronteira e as duas redes degradam de forma semelhante. A margem protege contra perturbações *pequenas*, regime em que o custo adicional de épocas da Adaline se justifica.



(a) teste com *holdout* 70/30



(b) degradação sob ruído

**Figura 3:** Teste da rede treinada sobre as amostras reservadas (a) e acurácia das duas redes sob ruído gaussiano nos atributos (b), com 200 repetições por nível; a faixa sombreada indica  $\pm 1$  desvio-padrão.

## 5 Conclusão

A Adaline treinada pela regra Delta classificou corretamente os 20 padrões da base B2, com pesos a 0,139 % do mínimo analítico, e o EQT decresceu monotonicamente de 28,26 para 3,71 em 704 épocas — o resíduo não nulo é inerente aos alvos bipolares com saída linear, não falha de aprendizado. O teste retornou 98,3 %  $\pm$  5,3 % sob *holdout* estratificado repetido e 100 % sob *leave-one-out*.

A taxa de aprendizagem mostrou-se crítica por um mecanismo menos óbvio que a divergência: todos os valores ficaram dentro do limite  $\eta < 0,3196$ , mas os maiores sofrem desajuste e convergem para um ponto fixo deslocado do mínimo, degradando a acurácia a 95,0 %. Frente ao Perceptron, a formulação por mínimos quadrados entregou margem 97 vezes maior, solução reproduzível e acurácia superior sob ruído. Como extensão natural, a mesma implementação pode ser aplicada a bases não linearmente separáveis, nas quais a Adaline degrada gradualmente enquanto o Perceptron não converge.

## Referências

- [1] WIDROW, B.; HOFF, M. E. **Adaptive switching circuits**. IRE WESCON Conv. Rec., p. 96–104, 1960.
- [2] HAYKIN, S. **Redes Neurais: Princípios e Prática**. 2. ed. Porto Alegre: Bookman, 2001.
- [3] SILVA, I. N.; SPATTI, D. H.; FLAUZINO, R. A. **Redes Neurais Artificiais para Engenharia e Ciências Aplicadas**. 2. ed. São Paulo: Artliber, 2016.
- [4] WIDROW, B.; LEHR, M. A. 30 years of adaptive neural networks. **Proc. IEEE**, v. 78, p. 1415–1442, 1990.

## Apêndice A — Código-fonte adaline\_b2.py

Programa utilizado para obter todos os resultados apresentados neste trabalho. Implementa a rede Adaline com a regra Delta (classe Adaline), o Perceptron simples usado na comparação da Seção 4 (classe Perceptron), as métricas de avaliação, os três protocolos de teste descritos na Seção 3 e a geração automática de todas as figuras. A execução padrão (python adaline\_b2.py) reproduz integralmente o experimento; os hiperparâmetros podem ser alterados pela linha de comando, por exemplo --eta 0.01 --eps 1e-7 --seed 7.

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  =====
5  Trabalho 05 - Classificacao de Padroes com a Rede Neural Adaline
6  Disciplina EL056 - Redes Neurais Artificiais - PPGEELT/UFU
7  Autor: Lucas Albino Martins (12622EEL006)
8  -----
9  Treina uma rede Adaline (ADaptive Linear NEuron, Widrow & Hoff, 1960) sobre a
10 base de dados B2, plota o erro quadratico total (EQT) durante o treinamento e
11 realiza o teste da rede treinada (holdout estratificado + leave-one-out).
12
13 Uso:
14     python adaline_b2.py                                # execucao completa (padrao)
15     python adaline_b2.py --eta 0.01 --eps 1e-7
16     python adaline_b2.py --base Basedados_B2.xls --figs figs --out resultados
17 =====
18 """
19
20 from __future__ import annotations
21
22 import argparse
23 import json
24 import os
25 from dataclasses import dataclass, field
26
27 import numpy as np
28 import pandas as pd
29 import matplotlib
30
31 if __name__ == "__main__":                                # em notebook, preserva o backend interativo
32     matplotlib.use("Agg")
33     import matplotlib.pyplot as plt
34
35 # -----
36 # Padronizacao visual das figuras
37 # -----
38 plt.rcParams.update({
39     "figure.dpi": 160,
40     "savefig.dpi": 160,
41     "font.size": 9,
42     "axes.grid": True,
43     "grid.alpha": 0.30,
44     "grid.linestyle": ":",
45     "axes.spines.top": False,
46     "axes.spines.right": False,
47     "legend.frameon": False,
48 })
49
50 COR_C1 = "#1b6ca8"    # classe +1
51 COR_C2 = "#c0392b"    # classe -1
52 COR_LIN = "#2c3e50"
53
54
55 # =====
56 # 1. REDE NEURAL ADALINE
57 # =====
```

```

58 @dataclass
59 class Adaline:
60     """Adaline de camada unica com regra Delta (Widrow-Hoff / LMS).
61
62     Convencao adotada (Silva, Spatti & Flauzino, 2016): a entrada e' aumentada
63     com  $x_0 = -1$ , de modo que  $w[0]$  representa o limiar de ativacao ( $\theta$ ) e o
64     potencial de ativacao e'  $u = W \cdot X = \sum_i (w_i x_i) - \theta$ .
65
66     Treinamento -> funcao de ativacao LINEAR ( $g(u) = u$ )
67     Operacao -> funcao de ativacao SIGNAL ( $y = \text{sgn}(u)$  em  $\{-1, +1\}$ )
68     """
69
70     eta: float = 0.0025 # taxa de aprendizagem
71     epsilon: float = 1e-6 # precisao requerida para  $|EQM(k) - EQM(k-1)|$ 
72     max_epocas: int = 10_000 # limite de seguranca
73     seed: int = 42 # semente dos pesos iniciais aleatorios
74
75     # --- atributos aprendidos -----
76     w: np.ndarray | None = None
77     w_inicial: np.ndarray | None = None
78     hist_eqt: list[float] = field(default_factory=list) # erro quadratico TOTAL
79     hist_eqm: list[float] = field(default_factory=list) # erro quadratico MEDIO
80     epocas: int = 0
81     convergiu: bool = False
82
83     # ----- utils
84     @staticmethod
85     def _aumenta(X: np.ndarray) -> np.ndarray:
86         """Concatena a entrada fixa  $x_0 = -1$  (associada ao limiar  $\theta$ )."""
87         X = np.atleast_2d(np.asarray(X, dtype=float))
88         return np.hstack([-np.ones((X.shape[0], 1)), X])
89
90     def potencial(self, X: np.ndarray) -> np.ndarray:
91         """Potencial de ativacao  $u = W \cdot X$  (saida linear, usada no treino)."""
92         return self._aumenta(X) @ self.w
93
94     def predict(self, X: np.ndarray) -> np.ndarray:
95         """Saida da rede em operacao:  $y = \text{sgn}(u)$  in  $\{-1, +1\}$ ."""
96         u = self.potencial(X)
97         return np.where(u >= 0.0, 1.0, -1.0)
98
99     def eqm(self, X: np.ndarray, d: np.ndarray) -> float:
100         """Erro quadratico medio  $E(w) = (1/p) * \sum_p (d_p - u_p)^2$ ."""
101         return float(np.mean((np.asarray(d, float) - self.potencial(X)) ** 2))
102
103     def eqt(self, X: np.ndarray, d: np.ndarray) -> float:
104         """Erro quadratico total  $EQT(w) = \sum_p (d_p - u_p)^2$ ."""
105         return float(np.sum((np.asarray(d, float) - self.potencial(X)) ** 2))
106
107     # ----- treino
108     def fit(self, X: np.ndarray, d: np.ndarray, verbose: bool = False) -> "Adaline":
109         """Treinamento supervisionado sequencial (padrao a padrao) pela regra Delta.
110
111          $w \leftarrow w + \eta * (d_p - u_p) * x_p$ 
112
113         Critério de parada:  $|EQM(atual) - EQM(anterior)| \leq \epsilon$ .
114         """
115         Xa = self._aumenta(X)
116         d = np.asarray(d, dtype=float).ravel()
117         rng = np.random.default_rng(self.seed)
118
119         # Pesos iniciais aleatorios pequenos (evita simetria e satura menos o erro)
120         self.w = rng.uniform(-0.5, 0.5, size=Xa.shape[1])
121         self.w_inicial = self.w.copy()
122
123         self.hist_eqt, self.hist_eqm = [], []
124         eqm_ant = float(np.mean((d - Xa @ self.w) ** 2))
125         self.hist_eqm.append(eqm_ant)
126         self.hist_eqt.append(eqm_ant * len(d))
127
128         self.epocas = 0
129         self.convergiu = False
130

```

```

131 while self.epocas < self.max_epocas:
132     # ---- uma epoca: percorre todos os p padroes de treinamento ----
133     for xa, alvo in zip(Xa, d):
134         u = float(xa @ self.w) # ativacao LINEAR
135         self.w += self.eta * (alvo - u) * xa
136
137     self.epocas += 1
138     eqm_atual = float(np.mean((d - Xa @ self.w) ** 2))
139     self.hist_eqm.append(eqm_atual)
140     self.hist_eqt.append(eqm_atual * len(d))
141
142     if verbose and self.epocas % 50 == 0:
143         print(f" epoca {self.epocas:5d} | EQM = {eqm_atual:.8f}")
144
145     if abs(eqm_atual - eqm_ant) <= self.epsilon:
146         self.convergiu = True
147         break
148     eqm_ant = eqm_atual
149
150     return self
151
152 # ----- reta/ODE
153 def reta_separacao(self, s1: np.ndarray) -> np.ndarray:
154     """Fronteira de decisao u = 0 no plano (s1, s2):
155     - theta + w1*s1 + w2*s2 = 0 => s2 = (theta - w1*s1) / w2
156     """
157     theta, w1, w2 = self.w[0], self.w[1], self.w[2]
158     return (theta - w1 * np.asarray(s1, float)) / w2
159
160 def __str__(self) -> str:
161     return (f"Adaline(theta={self.w[0]:+.4f}, w1={self.w[1]:+.4f}, "
162             f"w2={self.w[2]:+.4f}, epocas={self.epocas})")
163
164
165 # =====
166 # 2. PERCEPTRON (apenas para efeito comparativo na discussao)
167 # =====
168 @dataclass
169 class Perceptron:
170     eta: float = 0.01
171     max_epocas: int = 10_000
172     seed: int = 42
173     w: np.ndarray | None = None
174     epocas: int = 0
175     hist_erros: list[int] = field(default_factory=list)
176
177     def fit(self, X, d):
178         Xa = Adaline._aumenta(X)
179         d = np.asarray(d, float).ravel()
180         rng = np.random.default_rng(self.seed)
181         self.w = rng.uniform(-0.5, 0.5, size=Xa.shape[1])
182         self.hist_erros, self.epocas = [], 0
183         for _ in range(self.max_epocas):
184             erros = 0
185             for xa, alvo in zip(Xa, d):
186                 y = 1.0 if xa @ self.w >= 0 else -1.0 # ativacao DEGRAU
187                 if y != alvo:
188                     self.w += self.eta * (alvo - y) * xa
189                     erros += 1
190             self.epocas += 1
191             self.hist_erros.append(erros)
192             if erros == 0:
193                 break
194         return self
195
196     def predict(self, X):
197         return np.where(Adaline._aumenta(X) @ self.w >= 0, 1.0, -1.0)
198
199     def reta_separacao(self, s1):
200         return (self.w[0] - self.w[1] * np.asarray(s1, float)) / self.w[2]
201
202
203 # =====

```

```

204 # 3. METRICAS
205 # =====
206 def matriz_confusao(d, y):
207     """Retorna (VP, FN, FP, VN) considerando +1 como classe positiva."""
208     d, y = np.asarray(d).ravel(), np.asarray(y).ravel()
209     vp = int(np.sum((d == 1) & (y == 1)))
210     fn = int(np.sum((d == 1) & (y == -1)))
211     fp = int(np.sum((d == -1) & (y == 1)))
212     vn = int(np.sum((d == -1) & (y == -1)))
213     return vp, fn, fp, vn
214
215
216 def metricas(d, y) -> dict:
217     vp, fn, fp, vn = matriz_confusao(d, y)
218     n = vp + fn + fp + vn
219     acc = (vp + vn) / n if n else 0.0
220     prec = vp / (vp + fp) if (vp + fp) else 0.0
221     rec = vp / (vp + fn) if (vp + fn) else 0.0
222     f1 = 2 * prec * rec / (prec + rec) if (prec + rec) else 0.0
223     return {"acuracia": acc, "precisao": prec, "revocacao": rec, "f1": f1,
224            "VP": vp, "FN": fn, "FP": fp, "VN": vn}
225
226
227 def split_estratificado(X, d, frac_treino=0.7, seed=0):
228     """Particiona mantendo a proporcao original das duas classes."""
229     rng = np.random.default_rng(seed)
230     idx_tr, idx_te = [], []
231     for c in (-1, 1):
232         idx = np.where(np.asarray(d).ravel() == c)[0]
233         rng.shuffle(idx)
234         k = int(round(frac_treino * len(idx)))
235         idx_tr.extend(idx[:k])
236         idx_te.extend(idx[k:])
237     idx_tr, idx_te = np.array(idx_tr), np.array(idx_te)
238     rng.shuffle(idx_tr)
239     rng.shuffle(idx_te)
240     return X[idx_tr], d[idx_tr], X[idx_te], d[idx_te], idx_tr, idx_te
241
242
243 # =====
244 # 4. DADOS
245 # =====
246 def carregar_base(caminho="Basedados_B2.xlsx"):
247     """Le a base B2 e devolve (X, d, DataFrame)."""
248     df = pd.read_excel(caminho)
249     df.columns = [str(c).strip().lower() for c in df.columns]
250     X = df[["s1", "s2"]].to_numpy(float)
251     d = df["t"].to_numpy(float)
252     return X, d, df
253
254
255 # =====
256 # 5. FIGURAS
257 # =====
258 def fig_dispersao(X, d, path, modelo=None, titulo="Base de dados B2"):
259     fig, ax = plt.subplots(figsize=(4.4, 3.5))
260     m = d == 1
261     ax.scatter(X[m, 0], X[m, 1], c=COR_C1, marker="o", s=45,
262               edgecolor="white", linewidth=.7, label="Classe A ($t=+1$)", zorder=3)
263     ax.scatter(X[~m, 0], X[~m, 1], c=COR_C2, marker="s", s=45,
264               edgecolor="white", linewidth=.7, label="Classe B ($t=-1$)", zorder=3)
265     for i, (a, b) in enumerate(X, start=1):
266         ax.annotate(str(i), (a, b), textcoords="offset points", xytext=(5, 4),
267                   fontsize=5.8, color="#566573")
268     if modelo is not None:
269         s1 = np.linspace(X[:, 0].min() - .35, X[:, 0].max() + .35, 200)
270         ax.plot(s1, modelo.reta_separacao(s1), color=COR_LIN, lw=1.7,
271               label="Fronteira de decis\u00e3o", zorder=2)
272         ax.set_ylim(X[:, 1].min() - .4, X[:, 1].max() + .4)
273         ax.set_xlabel("$s_1$"); ax.set_ylabel("$s_2$")
274         ax.set_title(titulo, fontsize=10)
275         ax.legend(loc="upper left", fontsize=7.5)
276     fig.tight_layout(); fig.savefig(path); plt.close(fig)

```

```

277
278
279 def fig_erro(hist_eqt, hist_eqm, eps, path,
280             titulo="Erro quadr\u00e1tico total durante o treinamento"):
281     """Figura principal exigida no enunciado: EQT x \u00e9pocas + crit\u00e9rio de
    ↪ parada."""
282     fig, axs = plt.subplots(1, 2, figsize=(7.4, 3.0))
283     ep = np.arange(len(hist_eqt))
284
285     axs[0].plot(ep, hist_eqt, color=COR_C1, lw=1.7)
286     axs[0].axhline(hist_eqt[-1], color="#95a5a6", ls=":", lw=1.1)
287     axs[0].annotate(f"EQT final = {hist_eqt[-1]:.3f}",
288                    xy=(len(ep) * 0.45, hist_eqt[-1]), xytext=(0, 9),
289                    textcoords="offset points", fontsize=7.5, color="#566573")
290     axs[0].set_xlabel("\u00c9poca"); axs[0].set_ylabel("EQT")
291     axs[0].set_title("(a) erro quadr\u00e1tico total", fontsize=9)
292
293     delta = np.abs(np.diff(hist_eqm))
294     axs[1].semilogy(np.arange(1, len(delta) + 1), delta, color=COR_C2, lw=1.5)
295     axs[1].axhline(eps, color=COR_LIN, ls="--", lw=1.2,
296                  label=fr"\varepsilon = {eps:g}")
297     axs[1].set_xlabel("\u00c9poca"); axs[1].set_ylabel(r"$|EQM(k) - EQM(k-1)|$")
298     axs[1].set_title("(b) crit\u00e9rio de parada", fontsize=9)
299     axs[1].legend(fontsize=7.5)
300
301     fig.suptitle(titulo, fontsize=10)
302     fig.tight_layout(rect=(0, 0, 1, 0.94)); fig.savefig(path); plt.close(fig)
303
304
305 def fig_eta(curvas, path):
306     fig, ax = plt.subplots(figsize=(4.6, 3.3))
307     for eta, h in curvas.items():
308         ax.semilogy(np.arange(len(h)), h, lw=1.5, label=fr"\eta = {eta:g}")
309         ax.set_xlabel("\u00c9poca"); ax.set_ylabel("EQT (log)")
310         ax.set_title("Influ\u00eancia da taxa de aprendizagem", fontsize=10)
311         ax.legend(fontsize=7.5)
312     fig.tight_layout(); fig.savefig(path); plt.close(fig)
313
314
315 def fig_teste(Xtr, dtr, Xte, dte, modelo, path):
316     fig, ax = plt.subplots(figsize=(4.6, 3.5))
317     ax.scatter(Xtr[dtr == 1, 0], Xtr[dtr == 1, 1], c=COR_C1, marker="o", s=35,
318              alpha=.35, label="Treino $t=+1$")
319     ax.scatter(Xtr[dtr == -1, 0], Xtr[dtr == -1, 1], c=COR_C2, marker="s", s=35,
320              alpha=.35, label="Treino $t=-1$")
321     y = modelo.predict(Xte)
322     ok = y == dte
323     ax.scatter(Xte[ok & (dte == 1), 0], Xte[ok & (dte == 1), 1], c=COR_C1,
324              marker="o", s=95, edgecolor="k", linewidth=1.0, label="Teste $t=+1$",
325              ↪ zorder=4)
326     ax.scatter(Xte[ok & (dte == -1), 0], Xte[ok & (dte == -1), 1], c=COR_C2,
327              ↪ marker="s", s=95, edgecolor="k", linewidth=1.0, label="Teste $t=-1$",
328              ↪ zorder=4)
329     if np.any(~ok):
330         ax.scatter(Xte[~ok, 0], Xte[~ok, 1], facecolor="none", edgecolor="k",
331                  ↪ s=180, linewidth=1.6, label="Erro de classifica\u00e7\u00e3o",
332                  ↪ zorder=5)
333     s1 = np.linspace(0, 3.0, 200)
334     ax.plot(s1, modelo.reta_separacao(s1), color=COR_LIN, lw=1.7, zorder=2)
335     ax.set_xlim(-0.05, 3.0); ax.set_ylim(-0.25, 4.15)
336     ax.set_xlabel("$s_1$"); ax.set_ylabel("$s_2$")
337     ax.set_title("Teste da rede treinada (holdout 70/30)", fontsize=10)
338     ax.legend(fontsize=7, loc="upper left", ncol=2)
339     fig.tight_layout(); fig.savefig(path); plt.close(fig)
340
341
342 def fig_comparacao(X, d, net, perc, path):
343     """Adaline (solução de mínimos quadrados) x Perceptron (primeira solução)."""
344     fig, ax = plt.subplots(figsize=(4.6, 3.5))
345     m = d == 1
346     ax.scatter(X[m, 0], X[m, 1], c=COR_C1, marker="o", s=45,
347              ↪ edgecolor="white", linewidth=.7, label="Classe A ($t=+1$)", zorder=3)
348     ax.scatter(X[~m, 0], X[~m, 1], c=COR_C2, marker="s", s=45,

```

```

346         edgecolor="white", linewidth=.7, label="Classe B ($t=-1$)", zorder=3)
347     s1 = np.linspace(0, 3.0, 200)
348     ax.plot(s1, net.reta_separacao(s1), color=COR_LIN, lw=1.8, label="Adaline (LMS)")
349     ax.plot(s1, perc.reta_separacao(s1), color="#7f8c8d", lw=1.6, ls="--",
350             label="Perceptron (degrau)")
351     ax.set_xlim(-0.05, 3.0); ax.set_ylim(-0.25, 3.3)
352     ax.set_xlabel("$s_1$"); ax.set_ylabel("$s_2$")
353     ax.set_title("Fronteiras obtidas por Adaline e Perceptron", fontsize=10)
354     ax.legend(fontsize=7.5, loc="upper left")
355     fig.tight_layout(); fig.savefig(path); plt.close(fig)
356
357
358 def fig_ruído(sigmás, med_a, dp_a, med_p, dp_p, marg_a, marg_p, path):
359     """Degradação da acurácia sob ruído gaussiano nos
    → atributos."""
360     fig, ax = plt.subplots(figsize=(5.0, 3.5))
361     ax.plot(sigmás, med_a, color=COR_C1, lw=1.8, marker="o", ms=3.2, label="Adaline")
362     ax.fill_between(sigmás, med_a - dp_a, med_a + dp_a, color=COR_C1, alpha=.18)
363     ax.plot(sigmás, med_p, color=COR_C2, lw=1.8, ls="--", marker="s", ms=3.2,
364             label="Perceptron")
365     ax.fill_between(sigmás, med_p - dp_p, med_p + dp_p, color=COR_C2, alpha=.18)
366     ax.axvline(marg_a, color=COR_C1, lw=1.0, ls=":", alpha=.8)
367     ax.annotate(f"margem da Adaline = {marg_a:.3f}", xy=(marg_a, 0.845),
368                xytext=(5, 0), textcoords="offset points", fontsize=7,
369                color=COR_C1, va="center")
370     ax.set_xlabel("desvio-padrão do ruído $\sigma$")
371     ax.set_ylabel("acurácia média")
372     ax.set_ylim(0.83, 1.02)
373     ax.set_title("Robustez da fronteira a ruído nos atributos", fontsize=10)
374     ax.legend(fontsize=8, loc="lower left")
375     fig.tight_layout(); fig.savefig(path); plt.close(fig)
376
377
378 def fig_superfície(X, d, modelo, path):
379     """Regiões de decisão + valor do potencial de ativação u(s1,s2)."""
380     g1, g2 = np.meshgrid(np.linspace(-0.1, 3.1, 320), np.linspace(-0.3, 3.3, 320))
381     G = np.c_[g1.ravel(), g2.ravel()]
382     U = modelo.potencial(G).reshape(g1.shape)
383     fig, ax = plt.subplots(figsize=(4.6, 3.5))
384     cf = ax.contourf(g1, g2, U, levels=24, cmap="RdBu", alpha=.85)
385     ax.contour(g1, g2, U, levels=[0], colors=[COR_LIN], linewidths=1.8)
386     plt.colorbar(cf, ax=ax, label="potencial de ativação $u$")
387     m = d == 1
388     ax.scatter(X[m, 0], X[m, 1], c="w", edgecolor=COR_C1, marker="o", s=45,
    → linewidth=1.4)
389     ax.scatter(X[~m, 0], X[~m, 1], c="w", edgecolor=COR_C2, marker="s", s=45,
    → linewidth=1.4)
390     ax.set_xlabel("$s_1$"); ax.set_ylabel("$s_2$")
391     ax.set_title("Regiões de decisão da Adaline", fontsize=10)
392     fig.tight_layout(); fig.savefig(path); plt.close(fig)
393
394
395 # =====
396 # 6. EXPERIMENTOS
397 # =====
398 def experimento_completo(base, dir_figs, dir_out, eta, eps, seed):
399     os.makedirs(dir_figs, exist_ok=True)
400     os.makedirs(dir_out, exist_ok=True)
401     R: dict = {}
402
403     X, d, df = carregar_base(base)
404     R["n_amstras"] = int(len(d))
405     R["n_classe_pos"] = int(np.sum(d == 1))
406     R["n_classe_neg"] = int(np.sum(d == -1))
407     R["estatísticas"] = df[["s1", "s2"]].describe().round(4).to_dict()
408     print(f"[1] Base carregada: {len(d)} amostras, 2 atributos, "
409           f"{R['n_classe_pos']} (+1) / {R['n_classe_neg']} (-1)")
410     fig_dispersao(X, d, f"{dir_figs}/fig1_dispersao.png")
411
412     # --- 2. Treinamento com a base completa -----
413     net = Adaline(eta=eta, epsilon=eps, seed=seed).fit(X, d)
414     print(f"[2] Treinamento (base completa): {net} convergiu={net.convergiu}")
415     R["treino_completo"] = {

```

```

416     "eta": eta, "epsilon": eps, "seed": seed,
417     "w_inicial": net.w_inicial.round(4).tolist(),
418     "theta": round(float(net.w[0]), 4),
419     "w1": round(float(net.w[1]), 4),
420     "w2": round(float(net.w[2]), 4),
421     "epocas": net.epocas,
422     "eqt_inicial": round(net.hist_eqt[0], 4),
423     "eqt_final": round(net.hist_eqt[-1], 6),
424     "eqm_final": round(net.hist_eqm[-1], 6),
425     "convergiu": net.convergiu,
426 }
427 fig_erro(net.hist_eqt, net.hist_eqm, eps, f"{dir_figs}/fig2_eqt.png")
428 fig_dispersao(X, d, f"{dir_figs}/fig3_frenteira.png", modelo=net,
429               titulo="Frenteira de decis\u00e3o ap\u00f3s o treinamento")
430 fig_superficie(X, d, net, f"{dir_figs}/fig5_regioes.png")
431
432 y = net.predict(X)
433 R["desempenho_treino"] = metricas(d, y)
434 print(f"    acuracia na propria base de treinamento: "
435       f"{R['desempenho_treino']['acuracia']*100:.1f}%")
436
437 # --- 3. Sensibilidade a taxa de aprendizagem -----
438 curvas, tab_eta = {}, []
439 for e in (0.1, 0.05, 0.01, 0.0025, 0.001):
440     m = Adaline(eta=e, epsilon=eps, seed=seed).fit(X, d)
441     curvas[e] = m.hist_eqt
442     tab_eta.append({"eta": e, "epocas": m.epocas,
443                   "eqt_final": round(m.hist_eqt[-1], 4),
444                   "acuracia": round(metricas(d, m.predict(X))["acuracia"], 4),
445                   "convergiu": m.convergiu})
446     print(f"[3] eta={e:<7g} epocas={m.epocas:<5d} EQT={m.hist_eqt[-1]:9.4f} "
447          f"acc={tab_eta[-1]['acuracia']*100:5.1f}%")
448 R["tabela_eta"] = tab_eta
449 fig_eta(curvas, f"{dir_figs}/fig4_eta.png")
450
451 # --- 4. Teste: holdout estratificado 70/30 -----
452 linhas = []
453 for s in range(10):
454     Xtr, dtr, Xte, dte, itr, ite = split_estratificado(X, d, 0.7, seed=s)
455     m = Adaline(eta=eta, epsilon=eps, seed=seed).fit(Xtr, dtr)
456     mt = metricas(dte, m.predict(Xte))
457     linhas.append({"rodada": s + 1, "epocas": m.epocas,
458                  "acc_treino": round(metricas(dtr,
459 ↪ m.predict(Xtr))["acuracia"], 4),
460                  "acc_teste": round(mt["acuracia"], 4),
461                  "eqm_final": round(m.hist_eqm[-1], 6)})
462     if s == 0:
463         fig_teste(Xtr, dtr, Xte, dte, m, f"{dir_figs}/fig6_teste.png")
464         R["holdout_detalhe"] = {
465             "idx_treino": itr.tolist(), "idx_teste": ite.tolist(),
466             "theta": round(float(m.w[0]), 4), "w1": round(float(m.w[1]), 4),
467             "w2": round(float(m.w[2]), 4), "epocas": m.epocas,
468             "u_teste": np.round(m.potencial(Xte), 4).tolist(),
469             "y_teste": m.predict(Xte).astype(int).tolist(),
470             "d_teste": dte.astype(int).tolist(),
471             "metricas": mt}
472 R["holdout"] = linhas
473 accs = np.array([l["acc_teste"] for l in linhas])
474 R["holdout_resumo"] = {"media": round(float(accs.mean()), 4),
475                       "desvio": round(float(accs.std(ddof=1)), 4),
476                       "min": float(accs.min()), "max": float(accs.max())}
477 print(f"[4] Holdout 70/30 (10 rodadas): acuracia media = "
478       f"{accs.mean()*100:.1f}% +/- {accs.std(ddof=1)*100:.1f}%")
479
480 # --- 5. Teste: validacao cruzada leave-one-out -----
481 acertos, erros_loo = 0, []
482 for i in range(len(d)):
483     mask = np.ones(len(d), bool); mask[i] = False
484     m = Adaline(eta=eta, epsilon=eps, seed=seed).fit(X[mask], d[mask])
485     yi = float(m.predict(X[i:i+1])[0])
486     if yi == d[i]:
487         acertos += 1
488     else:

```

```

488         erros_loo.append(int(i))
489 R["loocv"] = {"acuracia": round(acertos / len(d), 4),
490             "acertos": acertos, "total": int(len(d)),
491             "indices_erro": erros_loo}
492 print(f"[5] Leave-one-out ({len(d)} particoes): acuracia = "
493       f"{acertos}/{len(d)} = {acertos/len(d)*100:.1f}%")
494
495 # --- 6. Operacao sobre padroes ineditos -----
496 novos = np.array([[0.50, 0.50], [0.90, 1.10], [1.50, 1.20],
497                  [1.60, 1.90], [2.40, 2.40], [2.90, 0.80]])
498 R["novos_padroes"] = [
499     {"s1": float(a), "s2": float(b), "u": round(float(u), 4), "y": int(yy)}
500     for (a, b), u, yy in zip(novos, net.potencial(novos), net.predict(novos))]
501 print("[6] Operacao sobre 6 padroes ineditos concluida")
502
503 # --- 7. Comparacao com o Perceptron -----
504 p = Perceptron(eta=eta, seed=seed).fit(X, d)
505
506 def margens(w):
507     """Distancia euclidiana com sinal de cada padrao a fronteira u = 0."""
508     return (Adaline._aumenta(X) @ w) / np.linalg.norm(w[1:])
509
510 marg_a, marg_p = margens(net.w), margens(p.w)
511 va = net.w[1:] / np.linalg.norm(net.w[1:])
512 vp = p.w[1:] / np.linalg.norm(p.w[1:])
513 R["perceptron"] = {"epocas": p.epocas,
514                  "acuracia": round(metricas(d, p.predict(X))["acuracia"], 4),
515                  "theta": round(float(p.w[0]), 4),
516                  "w1": round(float(p.w[1]), 4),
517                  "w2": round(float(p.w[2]), 4)}
518
519 R["margens"] = {
520     "adaline_min": round(float(np.min(np.abs(marg_a))), 4),
521     "perceptron_min": round(float(np.min(np.abs(marg_p))), 4),
522     "padrao_critico": int(np.argmax(np.abs(marg_a)) + 1),
523     "angulo_graus": round(float(np.degrees(np.arccos(np.clip(va @ vp, -1, 1)))),
524     ↪ 2)}
525
526 fig_comparacao(X, d, net, p, f"{dir_figs}/fig7_comparacao.png")
527 print(f"[7] Perceptron: {p.epocas} epocas, acuracia = "
528       f"{R['perceptron']['acuracia']*100:.1f}% | margem minima: "
529       f"Adaline {R['margens']['adaline_min']:.4f} x "
530       f"Perceptron {R['margens']['perceptron_min']:.4f}")
531
532 # --- 8. Estabilidade dos pesos frente a inicializacao -----
533 ws = []
534 for s in range(20):
535     m = Adaline(eta=eta, epsilon=eps, seed=s).fit(X, d)
536     ws.append([m.w[0], m.w[1], m.w[2], m.epocas, m.hist_eqm[-1]])
537 ws = np.array(ws)
538 R["estabilidade"] = {
539     "theta_media": round(float(ws[:, 0].mean()), 4),
540     "theta_desvio": round(float(ws[:, 0].std(ddof=1)), 5),
541     "w1_media": round(float(ws[:, 1].mean()), 4),
542     "w1_desvio": round(float(ws[:, 1].std(ddof=1)), 5),
543     "w2_media": round(float(ws[:, 2].mean()), 4),
544     "w2_desvio": round(float(ws[:, 2].std(ddof=1)), 5),
545     "epocas_media": round(float(ws[:, 3].mean()), 1),
546     "epocas_desvio": round(float(ws[:, 3].std(ddof=1)), 1),
547     "eqm_media": round(float(ws[:, 4].mean()), 6),
548     "eqm_desvio": round(float(ws[:, 4].std(ddof=1)), 8)}
549 print(f"[8] 20 inicializacoes distintas: EQM = {ws[:,4].mean():.6f} "
550       f"+/- {ws[:,4].std(ddof=1):.2e}")
551
552 # --- 9. Solucao analitica de minimos quadrados (referencia) -----
553 w_ot = np.linalg.pinv(Adaline._aumenta(X)) @ d
554 eqm_ot = float(np.mean((d - Adaline._aumenta(X) @ w_ot) ** 2))
555 R["solucao_analitica"] = {
556     "theta": round(float(w_ot[0]), 4), "w1": round(float(w_ot[1]), 4),
557     "w2": round(float(w_ot[2]), 4), "eqm": round(eqm_ot, 6),
558     "eqm_gradiente": round(net.hist_eqm[-1], 6),
559     "erro_relativo_pct": round(abs(net.hist_eqm[-1] - eqm_ot) / eqm_ot * 100, 4)}
560 print(f"[9] Solucao analitica: EQM = {eqm_ot:.6f} | obtida por gradiente: "
561       f"{net.hist_eqm[-1]:.6f} (desvio de "
562       f"{R['solucao_analitica']['erro_relativo_pct']:.3f}%")

```

```

560
561 # --- 10. Robustez da fronteira a ruído nos atributos -----
562 # As redes já treinadas (dados limpos) são submetidas a entradas
563 # corrompidas por ruído gaussiano; mede-se a queda da acurácia.
564 sigmas = np.linspace(0.0, 0.60, 13)
565 rep = 200
566 rng = np.random.default_rng(seed)
567 med_a, dp_a, med_p, dp_p = [], [], [], []
568 for sg in sigmas:
569     aa, pp = [], []
570     for _ in range(rep):
571         Xr = X + rng.normal(0.0, sg, size=X.shape) if sg > 0 else X
572         aa.append(metricas(d, net.predict(Xr))["acuracia"])
573         pp.append(metricas(d, p.predict(Xr))["acuracia"])
574         med_a.append(np.mean(aa)); dp_a.append(np.std(aa))
575         med_p.append(np.mean(pp)); dp_p.append(np.std(pp))
576 med_a, dp_a = np.array(med_a), np.array(dp_a)
577 med_p, dp_p = np.array(med_p), np.array(dp_p)
578 fig_ruído(sigmas, med_a, dp_a, med_p, dp_p,
579           R["margens"]["adaline_min"], R["margens"]["perceptron_min"],
580           f"{dir_figs}/fig8_ruído.png")
581
582 def sigma_ate(med, alvo=0.95):
583     """Maior sigma em que a acurácia média ainda se mantém acima de alvo."""
584     ok = sigmas[med >= alvo]
585     return round(float(ok.max()), 3) if len(ok) else 0.0
586
587 R["ruído"] = {
588     "sigmas": [round(float(v), 3) for v in sigmas],
589     "adaline_media": [round(float(v), 4) for v in med_a],
590     "perceptron_media": [round(float(v), 4) for v in med_p],
591     "repeticoes": rep,
592     "sigma_95_adaline": sigma_ate(med_a),
593     "sigma_95_perceptron": sigma_ate(med_p),
594     "acc_sigma_010": {"adaline": round(float(med_a[2]), 4),
595                      "perceptron": round(float(med_p[2]), 4)},
596     "acc_sigma_030": {"adaline": round(float(med_a[6]), 4),
597                      "perceptron": round(float(med_p[6]), 4)}
598 }
599 print(f"[10] Ruído: acurácia >= 95% ate sigma = {R['ruído']['sigma_95_adaline']}
600 ↪ "
601       f"(Adaline) x {R['ruído']['sigma_95_perceptron']} (Perceptron)")
602
603 with open(f"{dir_out}/resultados.json", "w", encoding="utf-8") as f:
604     json.dump(R, f, indent=2, ensure_ascii=False)
605
606 tab = df.copy()
607 tab["u"] = np.round(net.potencial(X), 4)
608 tab["y"] = net.predict(X).astype(int)
609 tab["acertou"] = (tab["y"] == tab["t"])
610 tab.to_csv(f"{dir_out}/saidas_treinamento.csv", index=False)
611
612 print(f"\nFiguras salvas em '{dir_figs}/', resultados em '{dir_out}/'.")
613 return R
614
615 # =====
616 def main():
617     ap = argparse.ArgumentParser(description="Adaline aplicado a base B2")
618     ap.add_argument("--base", default="Basedados_B2.xlsx")
619     ap.add_argument("--figs", default="figs")
620     ap.add_argument("--out", default="resultados")
621     ap.add_argument("--eta", type=float, default=0.0025)
622     ap.add_argument("--eps", type=float, default=1e-6)
623     ap.add_argument("--seed", type=int, default=42)
624     a = ap.parse_args()
625
626     print("=" * 72)
627     print(" Trabalho 05 - Adaline aplicado a classificacao de padroes (base B2)")
628     print("=" * 72)
629     experimento_completo(a.base, a.figs, a.out, a.eta, a.eps, a.seed)
630
631 if __name__ == "__main__":

```

```
632 |      main()
```